

# RabbitMQ 面试题总结（结合本仓库秒杀实战）

来源：[refactor/rabbitmq-message-queue](#) 分支秒杀异步化重构 — Redis Lua 校验 + RabbitMQ 异步落单 + DLQ 死信 + 有限重试。

## 一、基础概念

### 1. RabbitMQ 的核心模型是什么？

生产者 → **交换机(Exchange)** → 根据 **路由键(Routing Key)** 绑定 → **队列(Queue)** → 消费者。生产者不直接发消息到队列，而是发到交换机，由交换机按绑定规则分发。

### 2. 交换机有哪几种类型？各自适用场景？

| 类型      | 路由规则                      | 适用场景                                      |
|---------|---------------------------|-------------------------------------------|
| Direct  | 路由键完全匹配                   | 本仓库秒杀订单 <code>seckill.order</code> （精确路由） |
| Topic   | 路由键通配符匹配（* 一个词 / # 零或多个词） | 按主题/模块广播                                  |
| Fanout  | 忽略路由键，广播到所有绑定队列           | 全局通知                                      |
| Headers | 按消息 header 匹配（性能差，少用）     | 复杂多条件路由                                   |

### 3. 为什么消息不直接发队列，非要经过交换机？

解耦。生产者不关心队列存在与否、有几个消费者，只关心业务路由语义；队列和绑定关系可独立增删，扩展消费者、增加订阅方都不用改生产者代码。

## 二、消息可靠性（核心高频）

### 4. RabbitMQ 消息丢失有哪些场景？如何保证消息不丢？

三个环节：

| 环节           | 丢失原因      | 解决                                                                                                |
|--------------|-----------|---------------------------------------------------------------------------------------------------|
| 生产者 → Broker | 网络故障、发送失败 | <b>Confirm 模式</b> ( <code>publisher-confirm-type: correlated</code> ) + 失败重发                      |
| Broker 内部    | 宕机，消息在内存  | <b>队列持久化</b> ( <code>durable=true</code> ) + <b>消息持久化</b> ( <code>delivery_mode=2</code> ) + 集群镜像 |
| Broker → 消费者 | 消费者宕机/异常  | <b>手动 ACK</b> ，处理成功才确认，失败 NACK 重投                                                                 |

### 5. 什么是 ACK？AUTO / MANUAL 两种确认模式的区别？

- **AUTO**: Spring AMQP 默认, 方法正常返回自动确认; 抛异常自动 NACK 重投。开发者无感。
- **MANUAL** (本仓库采用): 需手动调用 `channel.basicAck(tag, false)` 确认, 或 `basicNack(tag, false, requeue)` 否定。完全控制投递确认时机, 失败时可决定重投还是入 DLQ。

关键: 手动 ACK 必须成功处理后才确认, 否则消息会一直 Unacked, 堆积在消费者连接上。

## 6. `basicNack` 的 `requeue` 参数 `true/false` 各代表什么?

- `requeue=true`: 消息重新投递回原队列, 等待再次消费 → 适合可恢复的临时故障。
- `requeue=false`: 消息被丢弃或进入 DLQ (若队列绑定了死信交换机) → 适合永久性失败。

本仓库: 失败先重试 (re-queue 带重试计数 header), 超过 3 次后直接发 DLQ 并 ACK 原消息, 避免无限循环。

## 7. 什么是死信队列 DLQ? 怎么配置?

消息成为死信的三种情况:

1. 消费者 `basicNack / basicReject` 且 `requeue=false`
2. 消息 TTL 过期
3. 队列达到最大长度, 超出部分被丢弃

配置: 原队列声明 `arguments.put("x-dead-letter-exchange", DLX) + x-dead-letter-routing-key`, 消息被拒绝/过期后自动转发到绑定了 DLX 的 DLQ。

本仓库: `seckill.order.queue` 失败超限 → 手动发到 `seckill.order.dlx` → `seckill.order.dlq`。

## 8. 为什么消费失败要有限重试 + 退避, 而不是无限重试?

无限重试的坏处:

- **死循环 CPU/IO**: 数据库持续打点, 压垮下游。
- **消息积压**: 消费线程卡在失败消息上, 后续正常消息无法消费。
- **雪上加霜**: 如果下游是 Redis/DB 短暂不可用, 重试风暴会拖垮恢复中的系统。

有限重试 (本仓库 2s/5s/10s 指数退避) 给下游喘息时间, 3 次仍失败说明大概率是永久性错误 (脏数据/代码 bug), 转入 DLQ 让人工介入。

## 9. 什么是消息幂等? 为什么 MQ 需要幂等? 本仓库怎么实现?

MQ 保证**至少一次投递** (At-least-once), 消费端可能重复收到同一条消息。幂等 = 同一条消息处理多次, 结果与处理一次一致。

本仓库两层幂等:

1. **消息级**: `orderId` 由 `RedisIdWorker` 预生成, 主键 `id` 冲突 → 重复 INSERT 失败回滚。
2. **业务级**: `tb_voucher_order` 唯一索引 (`user_id, voucher_id`), 不同 `orderId` 但同用户同券的重复请求被 DB 唯一约束拦截。

## 10. 秒杀场景为什么选 Lua + RabbitMQ 而不是直接在数据库扣库存?

- **Redis Lua 原子性**: 校验库存 + 扣减 + 一人一单 三件事在一个脚本内原子完成, 无锁、抗高并发。

- **数据库扛不住瞬时流量**：秒杀高峰 QPS 高，直接写库会打爆连接池。
- **削峰填谷**：RabbitMQ 作为缓冲，生产者快速返回，消费者按自身能力（`prefetch=1 + concurrency=3`）异步落单。
- **解耦**：秒杀校验与落单解耦，落单失败不影响用户下单响应。

---

## 三、性能与吞吐

### 11. 什么是 prefetch？怎么设置合理值？

`prefetch` = 消费者一次性向 Broker 预取的消息条数。

- **`prefetch=1`：公平分发**，一次只取 1 条，处理完再取下一条，避免慢消费者积压未处理消息（本仓库秒杀采用）。
- **`prefetch=N`（默认 250）**：吞吐高，但消息会批量堆积在消费者本地，慢消费者处理不完。

适合秒杀这种“每条消息处理耗时、需要公平”的场景用 **`prefetch=1`**；适合纯高速吞吐、处理极快的场景调大 `prefetch`。

### 12. 如何提升 RabbitMQ 消费吞吐？

1. 增加消费者并发（**`concurrency`**，本仓库 3，可调大）
2. 合理调大 `prefetch`
3. 多队列分区消费
4. 消息体精简、序列化高效（本仓库 JSON）
5. 避免每条消息都开事务/连库（批量落库）

### 13. 消息堆积怎么办？如何定位？

排查思路：

1. **看监控**：队列 Ready（待消费）数量持续上涨 = 生产快于消费。
2. **检查消费者**：是否卡在慢消息/失败重试上（Unacked 高 = 消费者处理慢或被阻塞）。
3. **临时扩容**：加消费者实例 / 调大 `concurrency`。
4. **积压处理**：紧急时关闭慢消费者、单独写脚本捞消息重放，或直接丢弃过期无效消息。

---

## 四、可靠性与高可用

### 14. 生产端如何保证消息不丢？（Confirm 模式）

```
spring:
  rabbitmq:
    publisher-confirm-type: correlated # 开启发布确认回调
    publisher-returns: true # 路由不可达时回调
```

生产者发消息后异步回调确认，未确认（**`nack`**）的消息记录补偿重发。本仓库目前未开 Confirm（MVP），生产环境应开启。

15. 消息持久化后就能保证不丢吗？

不能。持久化只保证 **Broker 宕机** 时消息从磁盘恢复，但不保证：

- 生产端到 Broker 之间的网络丢失（需 Confirm）
- 消费端拿到消息但未确认就宕机（需手动 ACK + 集群镜像）

持久化只是"减少丢失窗口"，完整可靠 = 持久化 + Confirm + 手动 ACK + 镜像队列。

16. RabbitMQ 集群高可用方案？

- **普通集群**：只复制队列元数据，队列数据单点 → 不满足高可用。
- **镜像队列**（classic mirror）：队列内容复制到多个节点，主节点宕机自动从从节点选举 → 高可用。
- **仲裁队列**（Quorum，新版推荐）：Raft 协议，多数派节点成功才算提交，数据更可靠，适合核心业务。

17. RabbitMQ vs Kafka 怎么选？

| 维度  | RabbitMQ            | Kafka       |
|-----|---------------------|-------------|
| 定位  | 通用消息中间件，路由灵活        | 分布式流平台 / 日志 |
| 吞吐  | 万级                  | 百万级         |
| 顺序性 | 单队列内有序，需特殊配置        | 分区内有序       |
| 可靠性 | 强大（Confirm/ACK/DLQ） | 强（ISR 副本）   |
| 适用  | 业务解耦、RPC、秒杀、任务队列    | 日志采集、埋点、流处理 |

秒杀异步落单这种**业务消息**适合 RabbitMQ；日志/事件流选 Kafka。

五、进阶与坑

18. 延迟消息怎么实现？

方案：

1. **TTL + DLX**：消息设 TTL，过期进死信交换机，绑定到目标队列 → 模拟延迟（本仓库 DLQ 机制可复用来做延迟）。
2. **延迟插件**（`rabbitmq_delayed_message_exchange`）：官方插件，原生支持延迟。
3. **消费者 sleep**：简单但阻塞消费线程（本仓库重试退避用了 `Thread.sleep`）。

19. 消息顺序如何保证？

RabbitMQ 默认不保证全局有序。方案：

1. **单队列 + 单消费者**：天然有序，但吞吐受限。
2. **同业务键路由到同一队列**：如按 `userId` 哈希路由，同一用户的消息进同一队列，队列内 FIFO。
3. **分区/仲裁队列**：配合顺序写。

秒杀场景通常不需要严格全局有序。

## 20. 消息重试时，怎么知道这是第几次？（本仓库实现）

利用消息 **header** 记录重试计数：

```
rabbitTemplate.convertAndSend(exchange, routingKey, order, msg -> {  
    msg.getMessageProperties().setHeader("x-retry-count", nextRetry);  
    return msg;  
});
```

消费时读 **x-retry-count**, **< 3** 则 NACK 重投（计数+1），**>= 3** 则发 DLQ。

## 21. 为什么本仓库消费者选择 **prefetch=1 + concurrency=3 + MANUAL ACK**？

- **prefetch=1**：每条消息公平处理，慢消息不阻塞其他消费者。
- **concurrency=3**：3 个消费线程并行，提升吞吐。
- **MANUAL ACK**：完全掌控确认时机——锁竞争失败、库存不足时抛异常 → 触发有限重试，而不是 Spring 默认的静默 ACK 导致丢单。

## 22. 秒杀订单消息消费失败的正确姿势？

本仓库的完整链路：

```
Consumer 捕获异常  
├─ 分布式锁获取失败 → 抛异常 → 触发重试  
├─ 库存不足 → 抛 RuntimeException → 触发重试  
└─ 重试计数 < 3 → 退避(2s/5s/10s)后重投  
    └─ 重试计数 >= 3 → 发 DLQ + ACK 原消息
```

要点：**失败必须让消息重回队列或进 DLQ，绝不能在 catch 里静默吞掉并 ACK**，否则订单丢失。

## 23. 同一个 key 加分布式锁的意义？会不会和 MQ 重复？

两者解决不同问题：

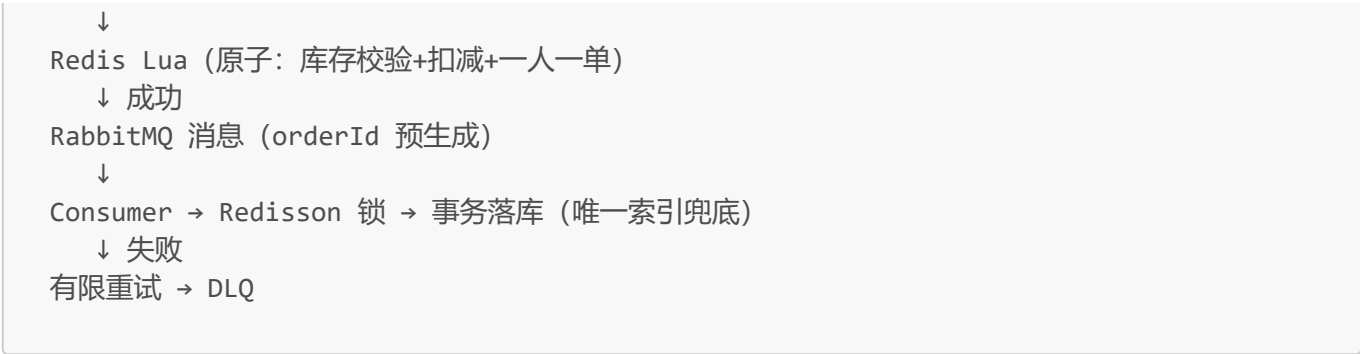
- **分布式锁（Redisson）**：保证同一用户的多条消息（可能来自多实例）**串行落库**，防止并发写同一用户订单。
- **MQ + 唯一索引**：保证**重复消息**只成功一次。
- 双保险：锁防并发，唯一索引防重复，互不替代。

---

## 六、秒杀高频八股

### 24. 秒杀系统的整体架构（本仓库）？

```
秒杀请求 → Nginx/网关  
↓  
Controller → seckillVoucher()
```



25. 秒杀中"一人一单"有几种实现层次？

| 层次                               | 实现       | 能防的问题                |
|----------------------------------|----------|----------------------|
| Redis Lua <code>sismember</code> | 内存级判重，原子 | 并发超卖、重复下单            |
| 分布式锁                             | 同用户消息串行  | 多实例并发落库              |
| DB 唯一索引                          | 数据库兜底    | Redis 数据丢失、锁失效后的最终防线 |

26. Redis 和 RabbitMQ 都可能在秒杀中宕机，如何兜底？

- **Redis 宕机**：Lua 校验失效 → 依赖 DB 唯一索引兜底（但可能超卖，需 DB 事务 + `stock>0` 条件更新）。
- **RabbitMQ 宕机**：秒杀响应会失败 → 需要消息重发机制 / 降级为同步落单（牺牲性能换可用）。

任何 MQ 架构都要考虑 **MQ 不可用的降级方案**（如同步写 + 补偿）。

## 七、实战陷阱

27. 手动 ACK 模式下最容易被忽视的坑？

1. **没配 `acknowledge-mode: manual`**：代码里手动 ACK，但容器是 AUTO → 行为不可控。
2. **`catch` 里吞异常**：业务失败不 NACK，消息被 ACK → 数据丢失。
3. **忘记 NACK**：抛异常但没处理，消息一直 Unacked，卡死消费线程。
4. **Unacked 消息堆积**：消费者处理慢，Broker 会积压 Unacked，需监控。

28. 为什么 `Thread.sleep` 做退避不优雅？生产怎么改进？

- 阻塞当前消费线程，消息大量失败时拖慢整个消费者。
- 改进：**TTL + 延迟交换机**（消息过段时间重回队列），或用延迟队列插件，让重试消息"排队等"，不阻塞活跃消费。

## 附：本项目 RabbitMQ 关键代码位置

| 文件                                              | 职责                            |
|-------------------------------------------------|-------------------------------|
| <code>config/RabbitMQConfig.java</code>         | 交换机/队列/DLX/DLQ 声明 + JSON 序列化  |
| <code>listener/VoucherOrderConsumer.java</code> | 消费 + 有限重试(2s/5s/10s) + DLQ 转发 |

| 文件                                        | 职责                             |
|-------------------------------------------|--------------------------------|
| service/impl/VoucherOrderServiceImpl.java | Lua 校验 → 发消息；消费端落库（锁 + 唯一索引兜底） |
| resources/seckill.lua                     | 原子库存校验/扣减/一人一单                 |
| docker-compose.server.yml                 | RabbitMQ 3.12 + 管理台(15670)     |